

TP : Manipulation des Commandes Artisan dans Laravel

Ce TP simplifié vise à vous familiariser avec les commandes Artisan de Laravel. Nous allons créer un projet très simple basé **uniquement sur le modèle User** (déjà présent dans Laravel par défaut). Toutes les données de test seront insérées via un **seeder** sur la table users.

1 : Création du Projet et Configuration

1. Créer le projet Laravel :

```
composer create-project laravel/laravel 013-tp-artisan
cd 013-tp-artisan
```

2 : Remplir la base avec un Seeder

Nous allons créer des utilisateurs de test directement dans la table users.

1. Modifier le fichier database/seeder/UsersTestSeeder.php :

```
class DatabaseSeeder extends Seeder
{
    use WithoutModelEvents;

    public function run()
    {
        User::factory(10)->create();
    }
}
```

2. Exécuter le seeder :

```
php artisan db:seed
```

3. Vérifier avec Tinker :

```
php artisan tinker
>>> App\Models\User::all()->pluck('name', 'id');
>>> exit
```

Vous devriez voir les 10 utilisateurs.

3 : Création de Commandes Artisan

3.1 Commande basée sur une classe – Lister les utilisateurs

1. Générer la commande :

```
php artisan make:command ListUsers
```

2. Modifier app/Console/Commands/ListUsers.php :

```
<?php

namespace App\Console\Commands;
```

```

use Illuminate\Console\Command;
use App\Models\User;

class ListUsers extends Command
{
    protected $signature = 'users:list';
    protected $description = 'Affiche la liste de tous les utilisateurs';

    public function handle()
    {
        $users = User::all();

        if ($users->isEmpty()) {
            $this->warn('Aucun utilisateur trouvé.');
```

3. Tester :

```
php artisan users:list
```

3.2 Commande basée sur une fermeture (Closure)

1. Ouvrir **routes/console.php** et ajouter :

```

use Illuminate\Support\Facades\Artisan;
use App\Models\User;

Artisan::command('users:count', function () {
    $count = User::count();
    $this->info("Nombre total d'utilisateurs : {$count}");
})->purpose('Affiche le nombre total d\'utilisateurs');
```

2. Tester :

```
php artisan users:count
```

3.3 Commande avec arguments et options

1. Générer :

```
php artisan make:command ShowUser
```

2. Modifier **app/Console/Commands/ShowUser.php** :

```

<?php

namespace App\Console\Commands;

use Illuminate\Console\Command;
use App\Models\User;

class ShowUser extends Command
```

```

{
    protected $signature = 'users:show {id} [--field=name : Champ à afficher (name ou email)]';
    protected $description = 'Affiche une information sur un utilisateur';

    public function handle()
    {
        $id = $this->argument('id');
        $field = $this->option('field');

        $user = User::find($id);

        if (!$user) {
            $this->error('Utilisateur non trouvé !');
            return;
        }

        $value = $user->{$field} ?? $user->name;
        $this->info("{ $field } de l'utilisateur ID { $id } : { $value }");
    }
}

```

3. Tester :

```

php artisan users:show 1
php artisan users:show 2 --field=email

```

3.4 Commande interactive (prompts)

1. Générer :

```

php artisan make:command CreateUserInteractive

```

2. Modifier **app/Console/Commands/CreateUserInteractive.php** :

```

<?php

namespace App\Console\Commands;

use Illuminate\Console\Command;
use App\Models\User;
use Illuminate\Support\Facades\Hash;

class CreateUserInteractive extends Command
{
    protected $signature = 'users:create';
    protected $description = 'Crée un nouvel utilisateur de façon interactive';

    public function handle()
    {
        $name = $this->ask('Nom de l\'utilisateur ?');
        $email = $this->ask('Email ?');
        $password = $this->secret('Mot de passe ?');

        $confirm = $this->confirm('Créer cet utilisateur ?', true);

        if ($confirm) {
            User::create([
                'name' => $name,
                'email' => $email,
                'password' => Hash::make($password),
            ]);
            $this->info('Utilisateur créé avec succès !');
        } else {

```

```
        $this->warn('Création annulée.');
```

3. Tester :

```
php artisan users:create
```

3.5 Commande avec choix multiple

1. Générer :

```
php artisan make:command DeleteUserInteractive
```

2. Modifier **app/Console/Commands/DeleteUserInteractive.php** :

```
<?php

namespace App\Console\Commands;

use Illuminate\Console\Command;
use App\Models\User;

class DeleteUserInteractive extends Command
{
    protected $signature = 'users:delete';
    protected $description = 'Supprime un utilisateur de façon interactive';

    public function handle()
    {
        $users = User::pluck('name', 'id')->toArray();
        if (empty($users)) {
            $this->warn('Aucun utilisateur à supprimer.');
```

3. Tester :

```
php artisan users:delete
```

4 : Vérification finale

Exécutez : `php artisan list`

Vous verrez toutes vos commandes personnalisées dans la liste.